Yash Rawat

CSE- DS

Company: ISS-Stoxx

Job Profile: Software Engineer

Offer Type: Internship + Placement

Internship Stipend: 60k

CTC: : 12.9 Base + 1.1 (PF and Gratuity) = 14L

Location: Nesco IT Park, Goregaon, Mumbai

Process Date: 05-08-2025, Completely Offline

Hello everyone!!! I hope you're all doing well and that your placement preparations are in full swing!
I'll be sharing my interview experience with ISS Stoxx, followed by some additional tips at the end. These include a few mistakes I made during the placement season, some good practices that helped me, and a few other useful insights. I highly recommend going through those as well — they can be just as important as the interview itself.

Placement Process:

A total of 3 rounds were conducted - 1 Online Assessment Round, 1 Tech Interview Round & Finally 1 HR Round. All were conducted on the same day 5th Aug. The CGPA Criteria for SDE was 8.5, later reduced to 8. It was further reduced to 7.5. May not be the case for next year if they don't want more applicants, so it is preferable to maintain a pointer of 8.5 and above. We were supposed to enter 2 preferences and mostly all of them were selected for their 1st preference. Some were asked personally whether they would like to sit for their interviews for the 2nd preference. So choose your preferences wisely.

1. **Resume based shortlisting**

My resume
The first round was a resume-based shortlisting. While the exact criteria were not disclosed, CGPA likely played a role.
Here are some key points based on my experience:

- Mention only those skills in your resume that you are genuinely confident and comfortable with. Including too many skills can lead to complex and unexpected questions during the interview if not fully prepared.
- Try to have at least one solid project. If the project is unique it's awesome but irrespective of if it is or not, focus on scalability and optimizations techniques.
- Ensure that all links in your resume are valid and clickable. Interviewers do check them. Also, make sure your projects have a proper README to give a quick overview of the project.
- Making your resume quantifiable using metrics or measurable impact is a good practice, though it is not strictly necessary.

- Try to align your resume with the job description or your selected preference. In our case, we were asked to choose two preferences (i chose SDE and Data Quality). Fortunately I was selected for the SDE role otherwise my resume didn't have any data science projects.. But that might not always be the case so make your resume smartly.
- Even if a company lists many technologies in the JD, you don't need to include all of them in your resume—focus on those that match your strengths and the role you're aiming for.

2. **Preplacement talk**

Although the process was initially planned to be offline, it was later conducted online at 5 PM on 4th August.

Pay close attention to the PPT. Note down the names and roles of the presenters—it helps during interactions. It's also a good idea to do some basic research on the company beforehand, like the founding year, CEO, and tagline, as such questions can be asked and showing awareness leaves a good impression.

You may ask thoughtful questions at the end of the PPT session to show engagement, though it's not mandatory. Alternatively, you can save your questions for the post-interview phase if that feels more comfortable.

3. **Online Assessment** [Around 160 candidates]

This round was held around 9:30 AM. It consisted of **70 questions in 90 minutes**, conducted via **Google Forms** at the college premises under the supervision of company professionals. The marking scheme was **+1 for correct answers** and **-0.25 for incorrect ones**. Only desktop calculators were allowed.

The test was divided into four sections: **Java, Python, SQL, and Aptitude**.

The Java section had **easy to medium-level questions**, mainly covering **OOPs, collections, multithreading exception handling, enums, etc**. Most questions involved code snippets where we had to predict the output. Although my primary language was C++, I had to prepare the **Java Collections Framework** separately. Previous years, topics like **streams, generics, concurrenthashmap** were also included, so it's advisable to prepare **Java thoroughly**. I remember I had solved 15/18 questions in this section. One or two were flukes.

The **Python** section included questions on **OOPs, MRO (Method Resolution Order), list comprehensions, Counter, stacks, sets, slicing**, and similar topics. Understanding the **internal workings** of Python is important—such as knowing when a **copy is created**, how **mutability** affects original data structures, and what happens when using **mutable default arguments** in functions. These concepts are frequently tested and can be tricky if not well understood.

The **SQL** section was straightforward. An input and a problem statement were given, and we had to choose the correct option—**no actual coding was required**. Questions were based on basic concepts like **JOINs, WHERE and BETWEEN clauses, and GROUP BY**. Preparing fundamental SQL queries is sufficient for this part.

I spent around 30–40 minutes on the Aptitude section after quickly completing the others. It was the toughest part of the assessment, with long and time-consuming questions, mostly based on IndiaBix. I avoided guessing and managed to attempt a fair number confidently. Topics included core areas like **profit and loss, interest, speed and distance, permutations, and data interpretation, along with less common ones like banker's discount and stocks.**

4. **Technical Round 1** [45 students]

I wasn't very confident about my OA performance, but I still began revising key concepts like Java Collections and reviewed my resume. The shortlist was announced at 12:15 PM, and I was delighted to see my name on the list. There were two interviewers in my panel, and they started asking questions simultaneously. The first question was a self-introduction, where I briefly mentioned my name, branch, projects, internship, committee role, and achievements—keeping it concise.

Next, I was asked to explain the four pillars of OOPs. I gave clear explanations with relevant examples. They followed up with questions on common interview topics like the difference between encapsulation and abstraction, and abstract classes vs. interfaces. I was specifically asked about constructors, instantiation, and whether these classes can have methods with their own implementation.Up to this point, I answered confidently. However, I got confused when asked if abstract classes and interfaces can have variables. I recalled that abstract classes can, but wasn't sure about interfaces due to less hands on practice with OOPs. I answered that it's possible but not commonly practiced—something I feel I shouldn't have said, as it didn't make much sense. These small gaps in lesser-practiced topics can create confusion, but the interview moved on smoothly after that.

They then asked if I was comfortable with Java Collections, and the same question was repeated for Python. Saying no directly could have been a red flag, so I answered honestly yet strategically. I mentioned that C++ is my primary language and that I'm very comfortable with DSA in C++ and OOPs in Java. I also said that while I may not have in-depth hands-on experience, I have a good understanding of Java Collections and know the basic concepts well. This seemed to satisfy them.

The interviewers continued with OOPs-related questions and asked about the different types of inheritance. When I mentioned multiple inheritance, they asked whether it's allowed in Java. I explained that it's not supported in the same way as in C++, but it can be achieved through interfaces. That led to a follow-up: why it is designed that way, what the diamond problem is, and how Java solves it.

Next, they asked whether an object can be used as a key in a HashMap. I recognized this question from the online assessment and replied that yes, it's possible. They then asked me to code it. Instead of creating a new example, I intentionally used the same one from the OA. I had been advised to try and control the flow of the interview, and since I had already explored this problem further (including using GPT), I was confident about the follow-up questions.

I used the following structure:

```
HashMap<Person, String> m = new HashMap<>();
m.put(p1, "yash");
m.put(p2, "yash");
```

This naturally led to the expected question: what is the size of the map?(because the value is same) I answered correctly—it's 2. The next question was how to make the size 1, even after inserting two objects.My first response was that using String keys with the same value would result in one entry due to string interning, but they asked me to solve it using the Person class itself. I took a moment and recalled that the .equals() method is used to compare keys. I explained that we need to override .equals() to ensure objects with the same attributes are treated as equal. The interviewer seemed happy with this answer. Though the answer should also have included the hashcode() function which I realised after the interview.

After that, the discussion moved to SQL. The questions were mostly basic, such as types of joins and the difference between left and right join. I explained that they are technically the same, and the only thing that changes is the order of the tables. I backed it up with an example, and the interviewer nodded in agreement.I was then asked how to filter results after using GROUP BY, to which I responded with the use of the HAVING clause. They also asked about the order of execution of SQL statements and whether it's possible to update a table using joins.I paused to think and said that while I haven't performed such operations directly, I believe it's possible if the update depends on the output of a join. I added that joins return a new table, so it might be a bit tricky. The interviewer pointed out that updates could depend on another table involved in the join, and that's when I realized the gap in my understanding.

I was also asked the difference between the PUT and POST methods. I gave the correct explanation, but since I hadn't actually used PUT in my projects, I became a bit unsure when asked if I was confident. I stuck to my answer but on questioning I accepted that I hadn't implemented it in any of the projects listed on my resume. In hindsight, if you're confident in your explanation, it's better to respond with certainty.

After this, I was asked to write a program to find the frequency of words in a sentence. I asked for permission to write it in C++, which they allowed. I used maps for storing frequencies and implemented the logic using stringstream. However, after writing the code, I realized I wasn't very clear about how stringstream works, though I still gave a rough explanation of its functionality. One part of the code included the operation map[word]++. The interviewer asked how I was incrementing the value if the word was not already present in the map. I explained that in C++, the default value for an int is 0 when a key is accessed for the first time, so the increment works. Then he asked about the default value for a string, and although I wasn't sure, I responded that I hadn't checked it specifically but assumed it would be null.

He then asked how to store the frequency using words like "one" or "two" instead of numbers. This caught me off guard. I said that it couldn't be done unless we predefined the mapping of numbers to words using another map. He still encouraged me to try, but I got confused. A few seconds later, he told me to leave it and moved on. I think the operation he asked for wasn't actually possible without an additional mapping.

He then asked about exception handling and this part went very smoothly. He gave a scenario with two catch blocks, one handling Exception and the next one IOException, along with a finally block. He asked how the code would behave if there was an exception during file reading. I replied that the code would not compile because the more specific exception, IOException, should come before the more general Exception. He then switched their order and asked again, and I answered correctly. After that, he asked what would happen if

there was a return statement in the try block. I said that the finally block would still execute. He further asked whether the return statement would still return a value, to which I responded yes.

The interview ended with me asking them a question. After coming out, I noticed that my interview had gone on for quite a long time—more than 45 minutes—while others were mostly done within 30 minutes, and some even in just 15–20 minutes. This made me nervous because it could mean many things. I also kept thinking about the few silly mistakes I made during the interview, like being unsure about a few answers as mentioned earlier and I had answered slightly wrong on the sql execution statements. Thankfully, friends came to the rescue again and helped calm me down. I then shifted my focus toward preparing for the HR round by going through common HR questions and the job description.

Tips:

As you might have noticed, my confidence was tested multiple times during the interview, and there were situations where the questions had logical answers, but I wasn't able to respond properly. Be prepared for this kind of pressure. According to a friend, one of my biggest weaknesses is being too honest in interviews—I tend to openly admit when I haven't done something, and I did that multiple times. It's better to avoid this.

Always try to relate your answer to something you know, even if you're not fully sure. Simply saying "I don't know" or "I haven't done this" isn't the best approach. Instead, respond strategically to keep the conversation in your control. Also, to understand your weak points and improve your delivery, I highly recommend doing mock interviews with friends—it really helps.

While explaining concepts, using examples is always the best approach as it makes your understanding clearer and more practical. While coding, speak out your thought process and approach unless you're specifically asked not to. This helps the interviewer understand how you think.

Also, keep a smile during the interview. Sometimes interviewers intentionally don't react to test your confidence. The way you enter the room, sit, and use gestures—all these non-verbal cues matter to some extent. While speaking, emphasize keywords intentionally. This can guide the interviewer to ask questions in areas you're comfortable with—for example, mentioning the diamond problem can lead the conversation in that direction.

5. **HR Round** [15 students]

The HR round took place around 6 PM and lasted approximately 30 minutes. The interviewers included the HR representative and the Managing Director of ISS.The first question was whether I recognized them, as they had presented the pre-placement presentation earlier. I answered correctly and acknowledged the importance of paying attention to such details.

Tip: Be prepared with another version of a non-technical introduction. In my case, I was asked only about my academic scores — SSC, HSC, CET percentile, and CGPA — and I gave a brief, to-the-point summary.

Next, I was asked why I chose computer science. I backed this up with a personal anecdote from my childhood and explained how my interest developed over time. I emphasized the genuine joy I feel when building real-life, working projects. I repeated this mindset throughout the interview to reflect my authenticity and passion for the field.

I was then asked about my family background. When they learned that my sisters had stable careers, they questioned why I wasn't pursuing a master's degree. I responded by again expressing my love for building applications and explained how I wanted to take the next step — to move beyond smaller-scale projects and work on larger systems that could help me grow and contribute more meaningfully. This response seemed to resonate with them.

They also asked how I planned to commute to work.

Then came a question to explain my favorite project, to which I described my internship project. However, this part brought back an old memory — in a previous HR round at MSCI, I was asked technical questions and didn't respond well, which may have contributed to my rejection. Similarly, here too, I was asked a technical question about the **scalability** of my internship project. My response wasn't very good. However, the Managing Director reassured me that he was more interested in my thought process, which helped a bit, but it still bugged me afterwards and reminded me of the earlier rejection.

They followed up by asking how I got the internship, what I liked most about it, and what differences I observed between working with college teammates versus an industry team.

Another important question came up — they asked how I'd feel if I had to work with a completely new tech stack or domain, and whether that would make all my prior learning go to waste. I confidently said **no**, explaining that no matter the stack, core concepts like APIs, databases, and clean code remain relevant. Every tech stack has its own strengths, and if the use case or project demands change, I'm ready to adapt.

The interview ended with me asking them some basic questions.

Everyone was asked to go home and results would be announced late evening.
Finally a list of 12 shortlisted individuals came out which had 8 SDEs.

Useful Tips:

- **Get used to rejections. Don't give up.**
  During the entire placement season, I was very sick (even hospitalized before it began), had severe side effects that caused excruciating joint pains, and still had to travel 1 hour daily for online assessments and interviews. I faced rejections in interviews from PhonePe, Barclays, Nomura, and in the HR round of MSCI. Even during the ISS process I was not well. But I kept pushing myself — just a little more every time.
- **Stay connected to your friends.**
  They help more than you can imagine — conduct mock interviews, build projects together, prepare in groups. It really helps in keeping the momentum and staying motivated.
- **Know all your projects well.**
  I've often heard people say, "It was a group project, and I didn't work on that part." Personally, I'd strongly recommend **not** saying that. You usually have 3-4 projects on your resume, and you're expected to know all of them thoroughly.
- **Some common resume/project questions:**
  - Why this project?
  - Why this tech stack? (know as much as you can about that technology)
  - Alternatives to the technologies used?

    - Your exact role?
    - What did you learn?
    - What are the drawbacks?
    - What can be improved?
    - Can it scale? How is the architecture?
- While answering, never talk negatively about alternatives. Instead, explain why **your chosen approach** suits your project **best**.
- Avoid saying I don't know unless you have zero idea about the question.
- Don't restrict your thoughts completely, convey them to the interviewer and clarify doubts. During some interviews out of nervousness I didn't say a few things which potentially could have been answers which I regretted; avoid it.
- **Don’t be fooled by simplicity.**
  After reading such experiences, you might feel that questions are easy/basic. **That’s a myth.**
  Many people get easy panels or familiar questions. But **rejections happen due to several other reasons** — timing, panel, tricky questions, or just not being the right fit.
  It’s not always about luck or coincidence either — some part of it is, but you should try to be **as prepared as possible**. Sometimes, when the panel has too many shortlisted candidates, they go tough. And **if you answer those tough questions, you automatically stand out.**
  The questions aren’t impossible — they’re just a little tricky and layered.
- **Never lose your calm. Always smile.**
  Communication plays a huge role. Your presence, gestures, tone — all matter. Highlight key words during your answers to guide the interviewer and keep the flow in your control.
- **Read the Job Description.**
  Align your strengths and weaknesses with the company’s values and goals.
- **Avoid last-minute prep.**
  Don’t rely on learning or reading something right before the interview.
- Have basic knowledge of **System design** not all companies ask them but phonepe, MSCI, Zeta, etc does.
- Prepare for a few HR questions beforehand.

Resources:

- Java and OOPs (refer at least the oops part): https://www.youtube.com/playlist?list=PL9gnSGHSqcnr_DxHsP7AW9ftq0AtAyYqJ
- Dbms: Complete DBMS in 1 Video (With Notes) || For Placement Interviews
- OS (Multithreading should be enough): Complete Operating Systems in 1 Shot (With Notes) || For Placement Interviews
- Indiabix for aptitude
- Sql top 50
- Striver Sheet for dsa is more than enough (unless you are appearing for phonepe).

Feel free to connect on LinkedIn.

All the very best!!! 😀